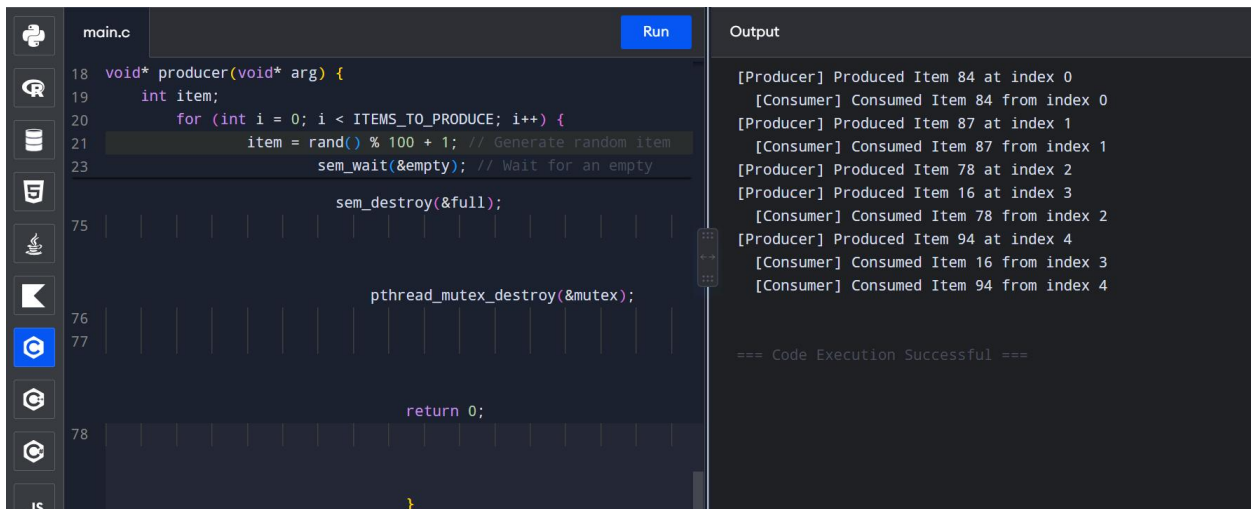


EXPERIMENT 39:

39. Write a C program to simulate the solution of Classical Process Synchronization Problem.



The screenshot shows a C program in a code editor with a dark theme. The code is for a Producer-Consumer problem simulation. It includes a `main.c` file with a `producer` function. The function generates random items and uses semaphores (`sem_wait`, `sem_post`) and a mutex (`pthread_mutex_lock`, `pthread_mutex_unlock`) to manage a shared buffer. The output window on the right shows the execution results, including item production and consumption at specific indices. The code execution is successful.

```
main.c Run
18 void* producer(void* arg) {
19     int item;
20     for (int i = 0; i < ITEMS_TO_PRODUCE; i++) {
21         item = rand() % 100 + 1; // Generate random item
22         sem_wait(&empty); // Wait for an empty
23         // ... (buffer management code) ...
24         sem_post(&full);
25     }
26     pthread_mutex_destroy(&mutex);
27     return 0;
28 }
```

Output

```
[Producer] Produced Item 84 at index 0
[Consumer] Consumed Item 84 from index 0
[Producer] Produced Item 87 at index 1
[Consumer] Consumed Item 87 from index 1
[Producer] Produced Item 78 at index 2
[Consumer] Consumed Item 78 from index 2
[Producer] Produced Item 16 at index 3
[Consumer] Consumed Item 16 from index 3
[Producer] Produced Item 94 at index 4
[Consumer] Consumed Item 94 from index 4

=== Code Execution Successful ===
```

PSEUDO CODE :

- Define buffer size N , shared circular buffer array, in and out pointers.
- Initialize POSIX semaphores:
 - empty semaphore initialized to buffer size N (tracks available empty spaces).
 - full semaphore initialized to 0 (tracks available filled items).
- Initialize mutex lock to handle critical section mutual exclusion.
- Producer Thread Function:
 - Loop K times: generate item.
 - Execute `sem_wait(&empty)` to block if buffer is full.
 - Acquire `pthread_mutex_lock(&mutex)`.
 - Place item in `buffer[in]`, update `in = (in + 1) % N`.
 - Release `pthread_mutex_unlock(&mutex)`.
 - Execute `sem_post(&full)` to signal an available item.
- Consumer Thread Function:
 - Loop K times:
 - Execute `sem_wait(&full)` to block if buffer is empty.
 - Acquire `pthread_mutex_lock(&mutex)`.
 - Retrieve item from `buffer[out]`, update `out = (out + 1) % N`.
 - Release `pthread_mutex_unlock(&mutex)`.
 - Execute `sem_post(&empty)` to signal empty slot available.
- Create producer and consumer threads using `pthread_create()`, wait for completion with `pthread_join()`, and clean up semaphores/mutex.

CODE:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <pthread.h>
```

```
#include <semaphore.h>
```

```
#include <unistd.h>
```

```
#define BUFFER_SIZE
```

```
5
```

```
#define
```

```
ITEMS_TO_PRODUC
```

```
E 5
```

```
int
```

```
buffer[BUFFER_SIZE];
```

```
int in = 0;
```

```
int out = 0;
```

```
sem_t empty;
```

```
sem_t full;
```

```
pthread_mutex_t mutex;
```

```
void* producer(void*  
arg) {
```

```
    int item;
```

```
    for (int i = 0; i <  
ITEMS_TO_PRODUC  
E; i++) {
```

```
        item = rand() %  
100 + 1; // Generate  
random item
```

```
sem_wait(&empty); //  
Wait for an empty slot
```

```
pthread_mutex_lock(&  
mutex); // Enter critical  
section
```

```
        buffer[in] = item;

        printf("[Producer]
Produced Item %d at
index %d\n", item, in);

        in = (in + 1) %
BUFFER_SIZE;

pthread_mutex_unlock(
&mutex); // Exit critical
section

        sem_post(&full); //
Signal that buffer has
new item

        sleep(1);

    }

    pthread_exit(NULL);
}
```

```
void* consumer(void*  
arg) {  
  
    int item;  
  
    for (int i = 0; i <  
ITEMS_TO_PRODUC  
E; i++) {  
  
        sem_wait(&full); //  
Wait for a full slot  
  
        pthread_mutex_lock(&  
mutex); // Enter critical  
section  
  
        item = buffer[out];  
  
        printf("  
[Consumer] Consumed  
Item %d from index  
%d\n", item, out);  
  
        out = (out + 1) %  
BUFFER_SIZE;
```

```
pthread_mutex_unlock(  
&mutex); // Exit critical  
section
```

```
sem_post(&empty); //  
Signal that buffer has  
empty space
```

```
    sleep(2);  
  
}  
  
pthread_exit(NULL);  
  
}
```

```
int main() {  
  
    pthread_t  
    prod_thread,  
    cons_thread;
```

```
// Initialize  
semaphores and mutex  
  
sem_init(&empty, 0,  
BUFFER_SIZE);  
  
sem_init(&full, 0, 0);  
  
pthread_mutex_init(&m  
utex, NULL);  
  
// Create Threads  
  
pthread_create(&prod_t  
hread, NULL, producer,  
NULL);  
  
pthread_create(&cons_t  
hread, NULL,  
consumer, NULL);  
  
// Join Threads
```

```
pthread_join(prod_thread,  
d, NULL);
```

```
pthread_join(cons_thread,  
d, NULL);
```

```
// Destroy
```

Semaphores and Mutex

```
sem_destroy(&empty);
```

```
sem_destroy(&full);
```

```
pthread_mutex_destroy(  
&mutex);
```

```
return 0;
```

```
}
```

INPUT :

(No direct user input

required; execution runs
with predefined buffer
size = 5 and items = 5)

OUTPUT:

[Producer] Produced

Item 84 at index 0

[Consumer] Consumed

Item 84 from index 0

[Producer] Produced

Item 87 at index 1

[Producer] Produced

Item 78 at index 2

[Consumer] Consumed

Item 87 from index 1

[Producer] Produced

Item 16 at index 3

[Consumer] Consumed

Item 78 from index 2

[Producer] Produced

Item 94 at index 4

[Consumer] Consumed

Item 16 from index 3

[Consumer] Consumed

Item 94 from index 4